A Coalgebraic and Resource-Sensitive Semantics for Tool-Augmented Agent Swarms

Author omitted for review

Draft of May 12, 2026

Abstract

Large language model agents are increasingly organised as systems of specialised components that use tools, maintain memory, delegate control, and interact through shared traces. Current engineering frameworks provide useful primitives such as tools, handoffs, guardrails, and tracing, but the semantic status of these primitives remains comparatively underdeveloped. This paper proposes a formal framework for tool-augmented agent swarms based on two ingredients: coalgebra, used to model state-based observable dynamics, and Subexponential Linear Logic (SELL), used to discipline resources, permissions, memory, budgets, and traces. We model an individual agent as a coalgebra whose observations trigger actions, tool calls, messages, handoffs, and state updates. A swarm is then represented as a coalgebra indexed by an interaction graph and equipped with provenance-carrying shared memory, an environment, and a resource ledger. The central semantic distinction is between raw executions, which a system may emit, and certified executions, which discharge labelled obligations for graph admissibility, tool permission, budget consumption, provenance, audit evidence, and answer support. SELL subexponentials and an explicit certification calculus provide a proof-theoretic layer for distinguishing persistent, shared, local, consumable, and auditable resources. The proposed semantics supports behavioural equivalence, compositional reasoning about handoffs, a certified-transition-system view, and resource-preservation properties for certified executions. An accompanying prototype trace checker validates JSON executions against the finite implemented fragment; for that fragment, accepted traces correspond to derivations in the certification calculus and generate reproducible artefacts for the paper. The complete prototype, conformance traces, and generated results are publicly archived at `https://github.com/cero1979/coalgebraic-agent-swarms` [46]. The contribution is not the claim that agents are merely automata, but rather a compositional account of contemporary agentic systems as observable, resource-governed, coalgebraic processes.

Keywords: coalgebra; agent swarms; multi-agent systems; tool-augmented agents; Subexponential Linear Logic; resource-aware reasoning; behavioural equivalence; verification.

1 Introduction

Recent agentic AI systems are no longer designed only as isolated predictors. They are increasingly assembled as interacting components: a planner delegates to specialised agents, agents invoke external tools, intermediate results are written to memory, and execution traces are used for debugging, evaluation, and governance. Frameworks such as AutoGen, LangGraph, and the OpenAI Agents

SDK make these patterns explicit by providing primitives for multi-agent conversation, tool use, handoffs, guardrails, and tracing [38, 41, 43, 44].

This engineering progress raises a semantic question. What is the mathematical object denoted by a tool-augmented agent or by a swarm of such agents? Standard accounts of agents as functions from percept histories to actions,

$$f : \mathsf{Obs}^* \to \mathsf{Act},$$

provide a useful extensional view, but they hide internal structure: memory, tool permissions, delegation, budgets, and proof traces. Conversely, implementation frameworks expose these structures operationally, but usually without an abstract semantics capable of supporting equivalence, compositionality, and resource-sensitive verification.

This paper develops a formal framework for this gap. The guiding thesis is:

> A tool-augmented agent swarm can be modelled as a coalgebraic state-based system whose transitions are governed by a resource-sensitive proof discipline.

Coalgebra provides the mathematics of states, observations, transitions, and behavioural equivalence [2, 3]. SELL provides a logical framework in which resources can be split into labelled zones with different structural rules, allowing one to distinguish persistent memory, shared knowledge, local agent state, tool permissions, budgets, and audit traces [1, 27, 28].

The main contribution is a compositional separation between *raw executions* and *certified executions*. Raw executions are behaviours of a coalgebra and may include unauthorised, unaffordable, or unaudited events. Certified executions are paths whose events discharge explicit labelled obligations: graph admissibility for communication and handoffs, tool permission, exact budget consumption, provenance-carrying memory extension, fresh audit evidence, and support for final claims. This separation is intended to make tracing and guardrails semantic rather than merely operational: a trace item is not only a log entry, but evidence tied to a certified transition and to later claims.

All results specific to the proposed framework are proved explicitly in the sequel. Standard background facts about coalgebras, finality, bisimulation, and subexponential proof systems are used only as cited background; the concrete final coalgebras and preservation properties needed for this paper are derived directly.

1.1 Contributions

The paper makes five conceptual and technical contributions.

1. It defines a coalgebraic semantics for tool-augmented agents in which observations produce events such as ordinary actions, tool calls, messages, handoffs, and final answers.
2. It extends the single-agent account to swarms by indexing coalgebras over an interaction graph and by adding shared memory, environment state, and a resource ledger.
3. It proposes a SELL-labelled subexponential signature and a concrete certification calculus for agentic resources, distinguishing persistent, shared, local, consumable, provenance-carrying, and trace-like assumptions.

4. It states semantic properties connecting raw dynamics and certified dynamics: existence of observable behaviours under final-coalgebra assumptions, bisimulation invariance, the induced certified transition system, and resource preservation for certified executions.
5. It provides a lightweight executable trace checker, an expanded suite of positive and negative traces, a checker–calculus adequacy result for the implemented fragment, and a minimal reproducible conformance scenario whose generated outputs are included as article artefacts and publicly archived at [46].

The final-coalgebra constructions used below are standard in spirit, and are not presented as new theorems of universal coalgebra. They are included to make explicit which observable behaviour is being compared. The novelty is the instantiation and coupling: contemporary agent operations are represented as observable coalgebraic events, while a SELL-labelled certification calculus separates raw executions from certified executions that preserve permissions, budgets, provenance, and audit evidence. The preservation theorem is therefore a local-to-global result: once each transition rule discharges local proof obligations, arbitrary finite certified compositions inherit global resource invariants. This gives more than a post-hoc log check: it identifies the obligations that a transition must satisfy before its effects may be treated as certified evidence.

1.2 Scope of the claim

The claim is deliberately modest. We do not claim that all AI agents are cellular automata, finite automata, or purely symbolic systems. Nor do we claim that SELL replaces probabilistic decision models such as MDPs or POMDPs, dynamic epistemic logics, separation logics, or type-and-effect systems. Rather, we propose a semantic interface: coalgebra captures the observable dynamics of agentic execution, while SELL constrains the use and movement of resources during that execution. Probabilistic choice, reinforcement learning, epistemic update, static effect checking, and separation-style memory ownership can be added as specialised layers.

2 Related Work

2.1 Coalgebra and state-based systems

Universal coalgebra was developed as a general theory of state-based systems, including automata, streams, transition systems, and dynamical systems [2, 3]. A coalgebra for an endofunctor F is a map $c : X \to F(X)$, where X is a state space and F specifies the observable shape of one computational step. Final coalgebras provide canonical domains of observable behaviour, while bisimulations provide coinductive criteria for behavioural equivalence.

Bisimulation is also central in process semantics and model checking, where it underlies equivalence checking and refinement arguments for transition systems [7, 8, 9]. This matters for agent swarms because two different decompositions of a task may be behaviourally equivalent even when their internal prompt state, local memory, or routing structure differs.

Coalgebraic modal logic gives logical languages for specifying and distinguishing coalgebraic behaviours; Pattinson's local-consequence account and the modular framework of Cîrstea, Kurz, Pat-

tinson, Schröder, and Venema are representative reference points [5, 6]. Probabilistic systems have also been studied coalgebraically. Sokolova surveys coalgebraic treatments of Markov chains, Markov processes, and probabilistic transition systems, emphasising behavioural equivalence for probabilistic dynamics [4]. Feys, Hansen, and Moss develop an algebraic and coalgebraic account of long-term values in Markov decision processes, connecting coalgebraic reasoning with optimal planning and policy improvement [19].

2.2 Coalgebraic approaches to agents and information update

Coalgebraic ideas have already appeared in epistemic and multi-agent settings. Baltag gives a coalgebraic semantics for epistemic programs and information updates [14]. Cîrstea and Sadrzadeh use coalgebraic semantics for information update in multi-agent systems without the usual change of model [15]. Carreiro, Gorín, and Schröder develop coalgebraic announcement logics, extending dynamic epistemic ideas to coalgebraic modal logic [16]. Hadzic and Chang apply coalgebra and coinduction to ontology-based multi-agent systems [17].

These works show that coalgebraic methods are already relevant to agents, knowledge, update, and decision. The present paper differs in focus: it targets contemporary tool-augmented agent swarms, where memory, tools, handoffs, traces, and resource budgets are first-class semantic entities.

2.3 Agentic AI frameworks and swarms

Classical multi-agent systems already treat autonomy, social ability, reactivity, proactiveness, communication, and organisation as defining concerns [22, 21, 23, 20, 24]. Agent communication languages such as KQML and FIPA ACL made message structure and communicative acts explicit [25, 26]. The present paper inherits this systems view, but its target is narrower: LLM-based agents whose execution logs expose tool calls, handoffs, shared state, and audit evidence.

The current engineering literature and documentation on LLM-based multi-agent systems emphasises role specialisation, conversation, tool use, orchestration, and delegation. AutoGen presents a multi-agent conversation framework for composing LLMs, tools, and humans in complex workflows [38]. Recent surveys describe LLM-based multi-agent systems in terms of workflows, infrastructure, communication, profiling, cooperation, and evaluation challenges [41, 42]. Tool-using language-model agents are also studied through reasoning-and-acting prompting and tool-use training regimes [39, 40]. OpenAI's Agents SDK describes agents as LLMs equipped with instructions and tools, with handoffs, guardrails, and tracing as core primitives [43]. LangGraph similarly treats agents as graph nodes and handoffs as routing operations carrying state updates or payloads [44].

These systems motivate the formal primitives used in this paper: agents, tools, handoffs, shared state, traces, and resource-sensitive control.

2.4 Subexponential Linear Logic

Linear logic is naturally resource-sensitive because assumptions are not freely reusable unless explicitly marked [1]. SELL refines this idea by decorating exponentials with subexponential labels.

These labels are organised by a preorder and may admit different structural rules such as weakening and contraction. Nigam and Miller show that subexponentials can represent locations of multisets of formulae and increase the algorithmic expressiveness of linear logic specifications [27]. Despeyroux, Olarte, and Pimentel compare Hybrid Linear Logic and SELL, showing how SELL represents modalities by labelled exponentials and an ordering among locations [28].

This makes SELL a natural candidate for modelling agentic resources: private memory, shared memory, tool permissions, budgets, and traces need not obey the same structural discipline.

The use of SELL here is deliberately proof-theoretic rather than merely taxonomic. Reusable permissions and trace facts are placed under structural subexponentials, while budget tokens and audit obligations remain linear. This distinction is difficult to express with a single global exponential, and it is the reason the framework uses subexponentials rather than ordinary linear logic alone.

2.5 Proof-carrying evidence and executable checkers

The executable part of this paper is closer to a proof checker or certifying algorithm than to an empirical benchmark. Certifying algorithms produce outputs together with witnesses that can be checked independently [10]; proof-carrying code accepts code only when accompanied by a checkable safety proof [11]. Certified proof checkers for SAT and related proof formats similarly distinguish generated evidence from the smaller algorithm that validates it [12]. Fully verified systems such as CompCert go further by mechanising large correctness proofs in a proof assistant [13]. The present artefact is intentionally at a lighter level: it specifies an abstract finite trace-checking function TraceCheck, proves adequacy for that function with respect to the certificate rules, and provides `prototype/checker.py` as a reproducible reference implementation. It does not claim that the Python implementation itself is mechanically verified.

2.6 Alternative formal tools

Several neighbouring formalisms solve different parts of the problem. Dynamic epistemic logic is a natural language for announcements, belief change, and information update [18]; it is less directly aimed at linear budgets or tool permissions. Separation logic gives precise reasoning principles for mutable heap ownership and frame conditions [29], but its native objects are heaplets rather than multi-agent handoff traces. Type-and-effect systems can statically approximate which tools or effects a program may use [30, 31], and algebraic effects give a semantic account of operations and handlers [32], but these approaches usually abstract away from run-time provenance and audit obligations. Provenance models such as W3C PROV and database provenance give a vocabulary for source history and derivation [33, 34, 35]; the present framework uses that intuition inside a resource-sensitive transition discipline rather than as a standalone metadata model. MDPs and POMDPs model optimal action under uncertainty [36, 37]; they are essential for policy and planning questions, but do not by themselves provide proof objects for resource use. Guardrails and tracing in engineering frameworks are operational checks and records; the aim here is to connect such checks to compositional obligations whose preservation follows across certified executions. The present framework should therefore be read as complementary to these approaches: coalgebra supplies observable dynamics, while SELL supplies labelled resource control over executions.

3 Preliminaries

3.1 Agents as external behaviours

Let Obs be a set of observations and Act a set of actions. A classical extensional representation of an agent is a function

$$f : \mathsf{Obs}^* \to \mathsf{Act},$$

where $f(w)$ is the action selected after the observation history w. This view is maximally abstract, but it hides the internal mechanisms by which the action is produced.

A state-based representation introduces an internal state space X, an output function, and a transition function. In deterministic Moore style one writes

$$o : X \to \mathsf{Act}, \qquad t : X \times \mathsf{Obs} \to X.$$

In deterministic Mealy style one writes

$$m : X \times \mathsf{Obs} \to \mathsf{Act} \times X.$$

Both are coalgebraic.

3.2 Coalgebras

Definition 1 (Coalgebra)**.** *Let $F : \mathbf{Set} \to \mathbf{Set}$ be an endofunctor. An F-coalgebra is a pair (X, c), where X is a set of states and*

$$c : X \to F(X)$$

is a transition-observation map.

For the Moore functor $F(X) = \mathsf{Act} \times X^{\mathsf{Obs}}$, a coalgebra is a map

$$c : X \to \mathsf{Act} \times X^{\mathsf{Obs}}.$$

It assigns to each state an observable action and a continuation for each possible observation.

For the Mealy functor $G(X) = (\mathsf{Act} \times X)^{\mathsf{Obs}}$, a coalgebra is a map

$$m : X \to (\mathsf{Act} \times X)^{\mathsf{Obs}}.$$

It assigns, for each state and observation, an action and a next state.

3.3 Final behaviour

Proposition 1 (Final Moore semantics)**.** *Let $F(X) = \mathsf{Act} \times X^{\mathsf{Obs}}$. The set*

$$B = \mathsf{Act}^{\mathsf{Obs}^*}$$

with coalgebra structure $\zeta : B \to \mathsf{Act} \times B^{\mathsf{Obs}}$ *defined by*

$$\zeta(b) = \big(b(\varepsilon),\, p \mapsto b_p\big), \qquad b_p(w) = b(pw),$$

is a final F*-coalgebra. Hence every* F*-coalgebra* $c : X \to \mathsf{Act} \times X^{\mathsf{Obs}}$ *induces a unique behaviour map*

$$[\![-]\!] : X \to \mathsf{Act}^{\mathsf{Obs}^*}.$$

Proof. Let (X, c) be an arbitrary F-coalgebra. Write

$$c(x) = \big(o(x), t_x\big), \qquad t_x : \mathsf{Obs} \to X.$$

For a word $w \in \mathsf{Obs}^*$, define the reached state x_w recursively by

$$x_\varepsilon = x, \qquad x_{pw} = \big(t_x(p)\big)_w.$$

Define $h : X \to B$ by

$$h(x)(w) = o(x_w).$$

We first show that h is a coalgebra morphism. For the output component,

$$h(x)(\varepsilon) = o(x_\varepsilon) = o(x).$$

For the transition component, fix $p \in \mathsf{Obs}$ and $w \in \mathsf{Obs}^*$. Then

$$h(x)_p(w) = h(x)(pw) = o(x_{pw}) = o((t_x(p))_w) = h(t_x(p))(w).$$

Thus $h(x)_p = h(t_x(p))$, and therefore

$$\zeta(h(x)) = \big(o(x), p \mapsto h(t_x(p))\big) = \big(\mathsf{Act} \times h^{\mathsf{Obs}}\big)(c(x)).$$

So h is a coalgebra morphism from (X, c) to (B, ζ).

It remains to prove uniqueness. Let $g : X \to B$ be any coalgebra morphism. Since g is a morphism, its output component gives

$$g(x)(\varepsilon) = o(x)$$

for every $x \in X$, and its transition component gives

$$g(x)(pw) = g(t_x(p))(w)$$

for all $p \in \mathsf{Obs}$ and $w \in \mathsf{Obs}^*$. We prove by induction on the length of w that $g(x)(w) = h(x)(w)$. If $w = \varepsilon$, then $g(x)(\varepsilon) = o(x) = h(x)(\varepsilon)$. If $w = pv$, then

$$g(x)(pv) = g(t_x(p))(v) = h(t_x(p))(v) = h(x)(pv),$$

where the middle equality follows from the induction hypothesis. Hence $g = h$. Therefore (B, ζ) is final. □

This map sends an internal state to its complete observable behaviour.

4 Tool-Augmented Agents as Coalgebras

The preceding account must be refined for modern agents, since their observable outputs are not merely ordinary actions. They may call tools, send messages, delegate control, emit a final answer, update memory, or emit audit evidence. We therefore distinguish between *raw events*, which may occur in an implementation trace, and *well-typed events*, which are compatible with the swarm interface.

Definition 2 (Local event interface)**.** *For an agent i, let*

$$\mathsf{Ev}_i = \mathsf{Act}_i + \mathsf{ToolCall}_i + \mathsf{Msg}_i + \mathsf{Handoff}_i + \mathsf{Answer}_i + \mathsf{TraceItem}_i.$$

The coproduct tags the observable kind of event. Thus a tool call is not identified with a tool name alone: it is an invocation carrying at least a tool identifier and a payload of arguments.

Definition 3 (Tool-augmented agent)**.** *A deterministic tool-augmented agent is a Mealy-style coalgebra*

$$c_i : X_i \to (\mathsf{Ev}_i \times X_i)^{\mathsf{Obs}_i}.$$

Equivalently, for $x \in X_i$ and $o \in \mathsf{Obs}_i$, one has $c_i(x)(o) = (e, x')$, where $e \in \mathsf{Ev}_i$ is the emitted event and $x' \in X_i$ is the next internal state.

The state space X_i may contain prompt state, scratchpad, local memory, retrieved context, pending goals, tool outputs, and other implementation-level data. Coalgebra abstracts from these implementation details while preserving the observable event-and-continuation structure.

Remark 1 (Probabilistic agents)**.** *A stochastic version is obtained by replacing deterministic continuations with a distribution functor:*

$$c_i : X_i \to \mathcal{D}(\mathsf{Ev}_i \times X_i)^{\mathsf{Obs}_i},$$

where $\mathcal{D}$ may be the finite-support distribution monad. This gives a coalgebraic interface to probabilistic transition systems, MDP-like dynamics, and stochastic policies. The present paper keeps the deterministic presentation because the resource theorem concerns certified finite executions.

4.1 Handoffs

A handoff transfers control, context, or responsibility from one agent to another. If V is the set of agents, a handoff from i to j carrying payload p is written

$$\mathsf{handoff}_{ij}(p) \in \mathsf{Handoff}_i.$$

At the local coalgebraic level this is just an event. At the swarm level it is admissible only if the interaction topology and the resource discipline permit it.

5 Agent Swarms

An agent swarm is not merely a product of isolated agents. Its essential structure is interaction: agents communicate, share memory, delegate control, compete for tools, and consume global resources. Formally, a swarm is a family of systems parameterised by the agent set and interaction graph. The three-agent example used later is only one finite instantiation; the definitions below are stated for an arbitrary set of agents V.

Definition 4 (Interaction graph)**.** *An interaction graph is a directed graph*

$$\Gamma = (V, E_\Gamma),$$

where V is the set of agents and $(i, j) \in E_\Gamma$ means that agent i may send a message or hand off control to agent j.

For a fixed graph Γ, define the raw event interface

$$\begin{aligned}\mathsf{RawEv}_\Gamma = &\sum_{i\in V} \mathsf{Act}_i + \sum_{i\in V} \mathsf{ToolCall}_i + \sum_{i,j\in V} (\mathsf{Msg}_{ij} + \mathsf{Handoff}_{ij}) \\ &+ \mathsf{MemUpdate} + \mathsf{EnvUpdate} + \mathsf{Answer} + \mathsf{TraceItem}.\end{aligned}$$

The well-typed graph-compatible events form a subset $\mathsf{Ev}_\Gamma \subseteq \mathsf{RawEv}_\Gamma$ obtained by retaining only those message and handoff events whose endpoints lie in E_Γ. Tool permissions and budgets are not built into this type: they are checked by the resource layer.

Shared memory is treated as a provenance-carrying finite partial map

$$M : \mathsf{Key} \rightharpoonup \mathsf{Val} \times \mathsf{Prov}.$$

If $M(k) = (v, p)$, then key k stores value v together with provenance record p, for instance the tool call, source identifier, and trace item by which the value entered the shared context. We write $M \sqsubseteq M'$ when M' extends M without changing any previously recorded binding. The deterministic core uses this append-only discipline for shared source keys; mutable scratchpads and overwritten local state remain inside the local state spaces X_i.

Let the global state space be

$$X_\Gamma = Y_\Gamma \times \mathsf{Res}, \qquad Y_\Gamma = \prod_{i\in V} X_i \times \mathsf{Mem} \times \mathsf{Env}.$$

The component Y_Γ contains local agent states, shared memory, and environment state; the Res component stores the resource ledger. We use a ledger of the form

$$R = (\beta, \pi, \tau, \kappa),$$

where $\beta : V \to \mathbb{N}$ gives the available linear budget per agent, $\pi \subseteq V \times \mathsf{Tool}$ records reusable tool permissions, $\tau \subseteq \mathsf{Trace}$ is the emitted audit log, and $\kappa \subseteq \mathsf{Claim} \times \mathsf{Key}$ records certified claims together with the shared-memory key that supports each claim. Thus certification is not just a Boolean flag on a sentence: it binds a claim to a source in the shared memory.

Definition 5 (Raw swarm coalgebra)**.** *Given an interaction graph Γ, a deterministic raw swarm coalgebra is a map*

$$C : X_\Gamma \to (\mathsf{RawEv}_\Gamma \times X_\Gamma)^{\mathsf{Obs}_\Gamma}.$$

For $x \in X_\Gamma$ and $o \in \mathsf{Obs}_\Gamma$, we write

$$x \xrightarrow[o\ C]{e} x'$$

when $C(x)(o) = (e, x')$.

The coalgebra describes the events an implementation may emit. It deliberately does not guarantee that every emitted event is permitted, affordable, or auditable. Those properties are captured by certification.

Definition 6 (Certified transition)**.** *Let Σ_Γ be a SELL subexponential signature for Γ. A certification relation is a judgement*

$$\Delta; \Theta \vdash_{\Sigma_\Gamma} e : (y, R) \rightsquigarrow (y', R'),$$

where Δ is the linear context, Θ is the subexponential context, $y, y' \in Y_\Gamma$, $R, R' \in \mathsf{Res}$, and $e \in \mathsf{RawEv}_\Gamma$. A raw transition $(y, R) \xrightarrow[o\ C]{e} (y', R')$ is certified if its non-ledger and ledger components satisfy such a judgement.

Thus the semantic object of interest is not just C, but the pair

$$(C, \vdash_{\Sigma_\Gamma}),$$

which separates possible dynamics from certified dynamics.

It is useful to name the certified subdynamics. For a raw swarm coalgebra C, define the certified transition relation

$$(y, R) \xRightarrow{e,o}_{\mathsf{cert}} (y', R')$$

iff $C(y, R)(o) = (e, (y', R'))$ and there is a derivation of $\Delta; \Theta \vdash_{\Sigma_\Gamma} e : (y, R) \rightsquigarrow (y', R')$. Thus certification turns the total raw coalgebra into a labelled transition system restricted to derivable steps. If every raw transition is certified, this restricted system is again total and can be regarded as a subcoalgebra on the same carrier. In the more realistic case, where some raw implementation steps are rejected, the certified behaviour is a partial LTS whose finite paths are precisely the executions used in the preservation theorem.

Proposition 2 (Final semantics for deterministic swarms)**.** *Let*

$$S_\Gamma(X) = (\mathsf{RawEv}_\Gamma \times X)^{\mathsf{Obs}_\Gamma}.$$

In **Set***, this functor has a final coalgebra isomorphic to*

$$\mathsf{RawEv}_\Gamma^{\mathsf{Obs}_\Gamma^+},$$

where Obs_Γ^+ is the set of non-empty finite observation words. Hence every raw swarm coalgebra $C : X_\Gamma \to S_\Gamma(X_\Gamma)$ induces a unique behaviour map

$$[\![-]\!]_\Gamma : X_\Gamma \to \mathsf{RawEv}_\Gamma^{\mathsf{Obs}_\Gamma^+}.$$

Proof. Let

$$B = \mathsf{RawEv}_\Gamma^{\mathsf{Obs}_\Gamma^+}.$$

For $b \in B$, define $\zeta(b) \in (\mathsf{RawEv}_\Gamma \times B)^{\mathsf{Obs}_\Gamma}$ by

$$\zeta(b)(o) = \big(b(o), b_o\big), \qquad b_o(w) = b(ow),$$

where $o \in \mathsf{Obs}_\Gamma$ is viewed as a one-letter word, $w \in \mathsf{Obs}_\Gamma^+$, and ow denotes concatenation. This is well typed because $ow \in \mathsf{Obs}_\Gamma^+$.

Let (X, C) be any S_Γ-coalgebra. For $x \in X$ and $o \in \mathsf{Obs}_\Gamma$, write

$$C(x)(o) = \big(e_C(x,o), t_C(x,o)\big).$$

For a non-empty word $u = o_1 \cdots o_n \in \mathsf{Obs}_\Gamma^+$, define states $x_0, \ldots, x_n$ and events $e_1, \ldots, e_n$ by

$$x_0 = x, \qquad C(x_{k-1})(o_k) = (e_k, x_k) \quad (1 \le k \le n).$$

Define $h : X \to B$ by

$$h(x)(o_1 \cdots o_n) = e_n.$$

This function records the last event emitted after processing the whole non-empty observation word. This does not discard trace information: for a concrete input word $u = o_1 \cdots o_n$, the emitted trace is recovered from the prefix evaluations

$$h(x)(o_1), \quad h(x)(o_1 o_2), \quad \ldots, \quad h(x)(o_1 \cdots o_n).$$

Thus the final behaviour $h(x) : \mathsf{Obs}_\Gamma^+ \to \mathsf{RawEv}_\Gamma$ is a prefix-indexed trace semantics. The notation returns one event per non-empty word because this is the standard final coalgebra for the deterministic Mealy functor; the finite event sequence is obtained by reading the behaviour along prefixes.

We prove that h is a coalgebra morphism. Fix $x \in X$ and $o \in \mathsf{Obs}_\Gamma$, and write $C(x)(o) = (e, x')$. The first component of $\zeta(h(x))(o)$ is

$$h(x)(o) = e.$$

For the second component, let $w \in \mathsf{Obs}_\Gamma^+$. Running the word ow from x first emits e and moves to x', and then running w from x' emits exactly the event $h(x')(w)$ at the end of the remaining word. Hence

$$h(x)_o(w) = h(x)(ow) = h(x')(w).$$

Since this holds for every w, $h(x)_o = h(x')$. Therefore

$$\zeta(h(x))(o) = \big(e, h(x')\big) = \big(\mathsf{RawEv}_\Gamma \times h\big)(C(x)(o)).$$

As this is true for every o, h is a coalgebra morphism.

Now let $g : X \to B$ be any coalgebra morphism. We show $g = h$. For a one-letter word o, the first component of the morphism equation gives

$$g(x)(o) = e_C(x,o) = h(x)(o).$$

For a word ow with $w \in \mathsf{Obs}_\Gamma^+$, the second component of the morphism equation gives

$$g(x)(ow) = g(t_C(x,o))(w).$$

We prove by induction on the length of $u \in \mathsf{Obs}_\Gamma^+$ that $g(x)(u) = h(x)(u)$. The length-one case was just proved. If $u = ow$ with $|w| \geq 1$, then

$$g(x)(ow) = g(t_C(x,o))(w) = h(t_C(x,o))(w) = h(x)(ow),$$

where the middle equality is the induction hypothesis applied to the shorter word w. Thus $g = h$, so (B, ζ) is final. □

Although $[\![x]\!]_\Gamma(u)$ returns the last event emitted after the observation word u, it does not discard trace information. Given a concrete input word $u = o_1 \cdots o_n$, the finite observable trace is recovered from the behaviour map by evaluating it on prefixes:

$$\big([\![x]\!]_\Gamma(o_1), [\![x]\!]_\Gamma(o_1 o_2), \ldots, [\![x]\!]_\Gamma(o_1 \cdots o_n)\big).$$

The audit component τ in the ledger records which of these emitted events carry certified trace identifiers. Thus coalgebraic behaviour supplies the observable event stream, while the certification layer determines which stream prefixes are auditable and resource-safe.

5.1 Bisimulation

Definition 7 (Swarm bisimulation)**.** *Let (X, C) and (Y, D) be two raw swarm coalgebras for the same functor S_Γ. A relation $R \subseteq X \times Y$ is a bisimulation if whenever $(x, y) \in R$, then for every observation $o \in \mathsf{Obs}_\Gamma$, the transitions*

$$C(x)(o) = (e, x'), \qquad D(y)(o) = (e', y')$$

satisfy $e = e'$ and $(x', y') \in R$.

Proposition 3 (Behavioural invariance)**.** *If $(x, y) \in R$ for some swarm bisimulation R, then*

$$[\![x]\!]_\Gamma = [\![y]\!]_\Gamma .$$

Proof. Let $u \in \mathsf{Obs}_\Gamma^+$. We prove by induction on the length of u that

$$[\![x]\!]_\Gamma(u) = [\![y]\!]_\Gamma(u)$$

whenever $(x, y) \in R$. If $u = o$ has length one, write

$$C(x)(o) = (e, x'), \qquad D(y)(o) = (e', y').$$

Since R is a bisimulation, $e = e'$. By the definition of the final behaviour map in Proposition 2,

$$[\![x]\!]_\Gamma(o) = e = e' = [\![y]\!]_\Gamma(o).$$

Now suppose $u = ow$, where $w \in \mathsf{Obs}_\Gamma^+$. Again write

$$C(x)(o) = (e, x'), \qquad D(y)(o) = (e', y').$$

Bisimulation gives $e = e'$ and $(x', y') \in R$. The final behaviour map satisfies

$$[\![x]\!]_\Gamma(ow) = \big[\![x']\!\big]_\Gamma(w), \qquad [\![y]\!]_\Gamma(ow) = \big[\![y']\!\big]_\Gamma(w).$$

By the induction hypothesis applied to $(x', y') \in R$ and the shorter word w,

$$[\![x']\!]_\Gamma(w) = [\![y']\!]_\Gamma(w).$$

Therefore $[\![x]\!]_\Gamma(ow) = [\![y]\!]_\Gamma(ow)$. Since this holds for every non-empty observation word, the two functions $[\![x]\!]_\Gamma$ and $[\![y]\!]_\Gamma$ are equal. □

For certified systems, the same definition applies directly to total coalgebras whose transitions are all certified. If certification restricts a raw coalgebra to a partial subtransition relation, the corresponding labelled-transition-system notion of bisimulation should be used instead. This is the formal basis for comparing different prompt decompositions, tool routings, or handoff policies by observable behaviour.

6 A SELL Layer for Agentic Resources

Coalgebra describes what transitions are possible. It does not, by itself, say whether a transition is permitted, affordable, auditable, or resource-safe. For that we add a proof-theoretic layer.

6.1 Subexponential signature

A SELL signature is a triple

$$\Sigma = (I, \preceq, U),$$

where I is a set of subexponential labels, $\preceq$ is a preorder on I, and $U \subseteq I$ is the set of labels admitting structural rules such as weakening and contraction. Labels outside U are linear in the relevant sense: their assumptions cannot be freely copied or discarded.

For an agent swarm, we use labels of the following form:

$$I_\Gamma = \{\mathsf{lin}, \mathsf{shared}, \mathsf{mem}, \mathsf{trace}, \mathsf{obl}\} \cup \{\mathsf{agent}_i, \mathsf{tool}_i, \mathsf{budget}_i \mid i \in V\}.$$

A minimal reusable set is

$$U_\Gamma = \{\mathsf{shared}, \mathsf{mem}, \mathsf{trace}\} \cup \{\mathsf{tool}_i \mid i \in V\},$$

while budget_i, lin, and obl remain non-structural. Thus tool permissions and recorded traces may be reused as evidence, whereas budget tokens and trace obligations are linear.

For the deterministic core, the preorder $\preceq_\Gamma$ is the smallest reflexive and transitive relation satisfying, for every $i \in V$,

$$\ell \preceq_\Gamma \mathsf{agent}_i \quad \text{for each } \ell \in \{\mathsf{shared}, \mathsf{mem}, \mathsf{trace}, \mathsf{lin}, \mathsf{obl}, \mathsf{tool}_i, \mathsf{budget}_i\}.$$

We read $\ell \preceq_\Gamma m$ as saying that resources stored at label ℓ may be inspected by a rule focused at label m, subject to the structural status of ℓ. No relation $\mathsf{agent}_j \preceq_\Gamma \mathsf{agent}_i$ is generated for $i \neq j$. Thus a rule for agent i may use shared facts, memory facts, trace evidence, its own tool permissions, and its own budget tokens, but it may not inspect another agent's private context unless the information has first been moved into a shared or trace label by an explicit event.

Label	Intended meaning	Structural status
mem	Persistent memory facts or summaries	reusable
shared	Shared blackboard or public context	reusable
trace	Emitted audit facts, usable as evidence	reusable
tool_i	Reusable permission for tools available to i	reusable
agent_i	Local assumptions of agent i	policy-dependent
budget_i	Consumable budget of calls, tokens, time, or money	linear
obl	Obligation to emit an audit item	linear
lin	Ordinary linear resources	linear

Table 1: A candidate subexponential discipline for tool-augmented agent swarms.

6.2 From SELL Sequents to Executable Certificates

We interpret SELL derivability operationally as a transition certificate over resource ledgers. To avoid leaving SELL as a merely decorative interface, this subsection fixes a small certification calculus. The calculus is intentionally shallow: it is not a complete proof-search procedure for SELL, but a family of admissible rule schemas whose premises are labelled by the subexponential zones in Table 1. Each schema produces one certified transition and one explicit ledger update.

The word "admissible" is used in the following restricted sense. A transition schema is admissible when its premises can be read as a SELL sequent in which reusable assumptions occur under labels in U_Γ, linear consumables occur in the ordinary linear context, and the conclusion consumes or preserves resources according to the structural status of those labels. For example, a tool-call certificate uses a reusable permission $!^{\mathsf{tool}_i}\mathsf{mayUse}(i,t)$, consumes a linear budget token at budget_i, and produces a persistent trace fact at trace. We do not claim a complete proof-search algorithm for arbitrary SELL formulae; the point is to give a precise SELL-labelled fragment whose rule instances are the proof objects checked below.

A judgement has the form

$$\Delta; \Theta \vdash_{\Sigma_\Gamma} e : (y, R) \rightsquigarrow (y', R'),$$

where Δ is a multiset of linear tokens and Θ is a labelled subexponential context. In the present calculus,

$$\Delta = \Delta_{\mathsf{lin}} \uplus \Delta_{\mathsf{obl}} \uplus \biguplus_{i \in V} \Delta_{\mathsf{budget}_i}, \qquad \Delta_{\mathsf{budget}_i} \ni \mathsf{budget}(i,n),\ n \in \mathbb{N},$$

$$\Theta = \Theta_{\mathsf{shared}} \uplus \Theta_{\mathsf{mem}} \uplus \Theta_{\mathsf{trace}} \uplus \biguplus_{i \in V} (\Theta_{\mathsf{agent}_i} \uplus \Theta_{\mathsf{tool}_i}).$$

The reusable components of Θ contain formulas such as $!^{\mathsf{tool}_i}\mathsf{mayUse}(i,t)$, $!^{\mathsf{shared}}\mathsf{source}(k,v,p)$, $!^{\mathsf{trace}}\mathsf{emitted}(q)$, and $!^{\mathsf{mem}}\mathsf{remembered}(k)$. Linear components of Δ contain consumable budget tokens and audit obligations. The bridge to the executable checker is obtained by reading each successful rule instance as a certificate object whose conclusion fixes the next ledger R'. Failed premises become checker rejections.

The ledger-to-context interpretation used in the fragment is:

$$\begin{aligned}\mathcal{I}(R, M) =& \{\mathsf{budget}(i, \beta(i))@\mathsf{budget}_i \mid i \in V\} \uplus \{!^{\mathsf{tool}_i}\mathsf{mayUse}(i, t) \mid (i, t) \in \pi\} \uplus \\ & \{!^{\mathsf{trace}}\mathsf{emitted}(q) \mid q \in \tau\} \uplus \{!^{\mathsf{shared}}\mathsf{source}(k, v, p) \mid M(k) = (v, p)\} \uplus \\ & \{!^{\mathsf{shared}}\mathsf{certified}(a, k) \mid (a, k) \in \kappa\}.\end{aligned}$$

The interpretation makes explicit why the calculus is resource-sensitive: permissions, source facts, trace facts, and certified-claim facts may be reused, but the budget token for i is a linear representative of the current numeric budget and is replaced rather than duplicated.

For a tool call by agent i using tool t, let q be a fresh trace identifier and let $r = \mathsf{cost}(t)$. The rule is sound only when $(i, t) \in \pi$, $\beta(i) \geq r$, and

$$R' = (\beta[i \mapsto \beta(i) - r], \pi, \tau \cup \{q\}, \kappa).$$

At the proof level this is represented by the schema

$$\frac{!^{\mathsf{tool}_i}\mathsf{mayUse}(i, t) \qquad \mathsf{budget}(i, \beta(i))@\mathsf{budget}_i \qquad q \notin \tau \qquad \beta(i) \geq \mathsf{cost}(t)}{\Delta; \Theta \vdash_{\Sigma_\Gamma} \mathsf{call}(i, t, q) : (y, R) \rightsquigarrow (y', R')}.$$

The budget premise is linear: the certificate consumes the old budget token and produces the reduced one. The freshness side condition ensures that the resulting trace item can be used as persistent audit evidence without discharging two obligations at once.

A more explicit sequent-style reading of the same schema is the following resource transformer:

$$\mathsf{budget}(i, n)@\mathsf{budget}_i, !^{\mathsf{tool}_i}\mathsf{mayUse}(i, t), \mathsf{obl}(q)@\mathsf{obl} \vdash \mathsf{budget}(i, n - \mathsf{cost}(t))@\mathsf{budget}_i \otimes !^{\mathsf{trace}}\mathsf{emitted}(q),$$

with the side condition $n \geq \mathsf{cost}(t)$ and $q \notin \tau$. This is the sense in which the rule is SELL-labelled: the permission is reusable, the trace fact becomes reusable evidence, while the budget and trace obligation are linear.

If the tool call writes a shared-memory key k with value v and provenance $p = (i, t, q)$, the memory-write schema requires freshness of the key and the corresponding audit item:

$$\frac{M(k) \text{ undefined} \qquad q \notin \tau \qquad p = (i, t, q)}{\Delta; \Theta \vdash_{\Sigma_\Gamma} \mathsf{writeShared}(i, k, v, p, q) : (y, R) \rightsquigarrow (y[M \mapsto M[k \mapsto (v, p)]], R')}.$$

Here $R' = (\beta, \pi, \tau \cup \{q\}, \kappa)$. This rule is the proof-theoretic counterpart of the append-only order $M \sqsubseteq M'$: once a key is available as source evidence, later transitions may cite it but may not silently overwrite it.

For a handoff from i to j, soundness requires $(i, j) \in E_\Gamma$, a payload satisfying the handoff policy, a fresh trace item q, and

$$R' = (\beta, \pi, \tau \cup \{q\}, \kappa).$$

The corresponding schema is

$$\frac{(i, j) \in E_\Gamma \qquad \mathsf{policy}_{ij}(p) \qquad q \notin \tau}{\Delta; \Theta \vdash_{\Sigma_\Gamma} \mathsf{handoff}_{ij}(p, q) : (y, R) \rightsquigarrow (y', R')}.$$

Messages use the same graph-and-audit discipline, without transferring control:

$$\frac{(i, j) \in E_\Gamma \qquad \mathsf{msgPolicy}_{ij}(m) \qquad q \notin \tau}{\Delta; \Theta \vdash_{\Sigma_\Gamma} \mathsf{message}_{ij}(m, q) : (y, R) \rightsquigarrow (y', R')}.$$

Here again $R' = (\beta, \pi, \tau \cup \{q\}, \kappa)$. For certification, a verifier may add a claim a supported by key k only when the key is already present in shared memory and the certifying agent is authorised:

$$\frac{i \in \mathsf{Certifiers} \qquad M(k) = (v, p) \qquad q \notin \tau}{\Delta; \Theta \vdash_{\Sigma_\Gamma} \mathsf{certify}(i, a, k, q) : (y, R) \rightsquigarrow (y, R')},$$

where $R' = (\beta, \pi, \tau \cup \{q\}, \kappa \cup \{(a, k)\})$. Finally, an answer rule can cite only already certified claims:

$$\frac{\forall a \in A\ \exists k_a.\, (a, k_a) \in \kappa \qquad q \notin \tau}{\Delta; \Theta \vdash_{\Sigma_\Gamma} \mathsf{answer}(i, A, q) : (y, R) \rightsquigarrow (y, R')},$$

where $R' = (\beta, \pi, \tau \cup \{q\}, \kappa)$. These six schemas are the minimal calculus used in the preservation theorem and in the prototype: tool calls spend budget, memory writes create provenance-carrying source keys, handoffs and messages enforce the graph policy, certification binds claims to those keys, and answers use only certified evidence.

The six schemas have a uniform SELL-labelled resource-transformer reading. In the following display, formulas of the form $!^{\ell} A$ are reusable when $\ell \in U_\Gamma$, whereas budget tokens and audit obligations remain linear. The side conditions are exactly those stated in the operational schemas: known endpoints, graph edges, payload policies, source-key freshness, trace freshness, certifier authority, source availability, and budget sufficiency.

$$\begin{aligned}
\mathsf{call}: \quad & \mathsf{budget}(i, n)@\mathsf{budget}_i, !^{\mathsf{tool}_i}\mathsf{mayUse}(i, t), \mathsf{obl}(q)@\mathsf{obl} \vdash \\
& \mathsf{budget}(i, n - \mathsf{cost}(t))@\mathsf{budget}_i \otimes !^{\mathsf{trace}}\mathsf{emitted}(q) \\
\mathsf{writeShared}: \quad & \mathsf{freshKey}(k), \mathsf{prov}(p, i, t, q), \mathsf{obl}(q)@\mathsf{obl} \vdash \\
& !^{\mathsf{shared}}\mathsf{source}(k, v, p) \otimes !^{\mathsf{trace}}\mathsf{emitted}(q) \\
\mathsf{handoff}: \quad & !^{\mathsf{shared}}\mathsf{edge}_\Gamma(i, j), !^{\mathsf{agent}_i}\mathsf{handoffPolicy}_{ij}(p), \mathsf{obl}(q)@\mathsf{obl} \vdash \\
& !^{\mathsf{trace}}\mathsf{handoff}(i, j, q) \otimes !^{\mathsf{trace}}\mathsf{emitted}(q) \\
\mathsf{message}: \quad & !^{\mathsf{shared}}\mathsf{edge}_\Gamma(i, j), !^{\mathsf{agent}_i}\mathsf{msgPolicy}_{ij}(m), \mathsf{obl}(q)@\mathsf{obl} \vdash \\
& !^{\mathsf{trace}}\mathsf{message}(i, j, q) \otimes !^{\mathsf{trace}}\mathsf{emitted}(q) \\
\mathsf{certify}: \quad & !^{\mathsf{agent}_i}\mathsf{mayCertify}(i), !^{\mathsf{shared}}\mathsf{source}(k, v, p), \mathsf{obl}(q)@\mathsf{obl} \vdash \\
& !^{\mathsf{shared}}\mathsf{certified}(a, k) \otimes !^{\mathsf{trace}}\mathsf{emitted}(q) \\
\mathsf{answer}: \quad & \bigotimes_{a \in A} !^{\mathsf{shared}}\mathsf{certified}(a, k_a), \mathsf{obl}(q)@\mathsf{obl} \vdash . \\
& !^{\mathsf{trace}}\mathsf{answered}(A, q) \otimes !^{\mathsf{trace}}\mathsf{emitted}(q)
\end{aligned}$$

The cut between the certification conclusion and an answer premise is harmless because certified-claim facts live in the reusable shared zone. By contrast, each audit obligation remains linear, so the same audit item cannot discharge two auditable steps. This is the concrete proof-theoretic reason why certified claims may be cited repeatedly, but trace obligations must be fresh at every auditable step.

Definition 8 (Local soundness of certificates)**.** *A family of SELL transition rules is locally sound for* Γ *if each certified step*

$$e : (y, R) \rightsquigarrow (y', R')$$

with shared-memory components M, M'*,* $R = (\beta, \pi, \tau, \kappa)$*, and* $R' = (\beta', \pi', \tau', \kappa')$ *satisfies the following conditions:*

1. *graph condition: if* e *is a message or handoff from* i *to* j*, then* $(i, j) \in E_\Gamma$*;*

2. *permission condition: if e is a tool call by i to tool t, then $(i,t) \in \pi$;*

3. *shared-memory condition: $M \sqsubseteq M'$, and every newly allocated key in $\mathrm{dom}(M') \setminus \mathrm{dom}(M)$ carries provenance containing the event and its trace item;*

4. *budget condition: for each agent i,*

$$\beta'(i) = \beta(i) - c_i(e) + \rho_i(e),$$

where $c_i(e) \geq 0$ is the certified cost and $\rho_i(e) \geq 0$ is an explicitly certified replenishment;

5. *non-negativity condition: $\beta'(i) \in \mathbb{N}$ for every $i \in V$;*

6. *trace condition: $\tau \subseteq \tau'$, and if e is auditable, then there exists a trace item $q_e \in \tau' \setminus \tau$;*

7. *certification monotonicity: $\kappa \subseteq \kappa'$;*

8. *certification condition: every pair $(a,k) \in \kappa' \setminus \kappa$ is supported by a source key $k \in \mathrm{dom}(M)$ already present before the certification step;*

9. *answer condition: if e is a final answer citing claims K_e, then for every $a \in K_e$ there is some key k such that $(a,k) \in \kappa$.*

Operational rule	SELL-labelled resources	Local obligation discharged
$\mathsf{call}(i,t,q)$	$!^{\mathsf{tool}_i}\mathsf{mayUse}(i,t)$, $\mathsf{budget}(i,n)@\mathsf{budget}_i$, fresh trace obligation	permission, budget decrement, non-negativity, fresh audit item
$\mathsf{writeShared}(i,k,v,p,q)$	shared-memory source key, provenance term p, fresh trace obligation	append-only memory extension, provenance recording, fresh audit item
$\mathsf{handoff}_{ij}(p,q)$	agent-local payload policy, graph fact, fresh trace obligation	graph preservation, payload admissibility, fresh audit item
$\mathsf{message}_{ij}(m,q)$	message policy, graph fact, fresh trace obligation	graph preservation, message admissibility, fresh audit item
$\mathsf{certify}(i,a,k,q)$	certifier authority, $!^{\mathsf{shared}}\mathsf{source}(k,v,p)$, fresh trace obligation	claim-source binding, certification monotonicity, fresh audit item
$\mathsf{answer}(i,A,q)$	certified claim-source pairs $(a,k_a) \in \kappa$, fresh trace obligation	answer safety and traceability

Table 2: Executable certificate rules and the local soundness obligations they discharge.

Lemma 1 (Local soundness of the certification calculus)**.** *Every instance of the six certificate schemas above that satisfies its stated premises satisfies the corresponding clauses of Definition 8.*

Proof. The proof is by case analysis on the last rule used to derive the certificate. For $\mathsf{call}(i,t,q)$, the premise $!^{\mathsf{tool}_i}\mathsf{mayUse}(i,t)$ is interpreted in the ledger as $(i,t) \in \pi$, the budget premise gives $\beta(i) \geq \mathsf{cost}(t)$, and the conclusion sets $\beta'(i) = \beta(i) - \mathsf{cost}(t)$ while leaving other budgets unchanged. Hence the permission, budget, and non-negativity clauses hold. The side condition $q \notin \tau$ and the update $\tau' = \tau \cup \{q\}$ give trace freshness, while memory and certified claims are unchanged.

For $\mathsf{writeShared}(i,k,v,p,q)$, freshness of k and the update $M' = M[k \mapsto (v,p)]$ give $M \sqsubseteq M'$. The premise fixing $p = (i,t,q)$, or more generally a provenance term containing the event and trace item,

gives the shared-memory provenance clause. Again $q \notin \tau$ and $\tau' = \tau \cup \{q\}$ give trace freshness, while budgets and certified claims are unchanged.

For $\mathsf{handoff}_{ij}(p, q)$, the premise $(i, j) \in E_\Gamma$ is exactly the graph condition. The payload-policy premise states that the handoff payload is admissible. The trace update gives a fresh audit item, and no budget, memory, or certification component is changed.

For $\mathsf{message}_{ij}(m, q)$, the same graph premise gives the graph condition for messages, the message-policy premise states admissibility of the message payload, and the trace update gives a fresh audit item. Budgets, memory, and certified claims are unchanged.

For $\mathsf{certify}(i, a, k, q)$, the certifier-authority premise authorises the rule instance, and the premise $M(k) = (v, p)$ ensures that the source key is already present before the claim is added. The conclusion updates κ to $\kappa \cup \{(a, k)\}$, so certification is monotone and every newly added claim has a shared-memory source. Trace freshness follows from the same side condition and update as above.

For $\mathsf{answer}(i, A, q)$, the premise $\forall a \in A\, \exists k_a.(a, k_a) \in \kappa$ is exactly the answer-safety condition. The conclusion leaves budgets, memory, and certifications unchanged and adds the fresh audit item q. These cases exhaust the calculus, proving the lemma. □

The purpose of Lemma 1 is not to establish a new metatheorem of SELL. Its role is *architectural*: it is the precise bridge between the six labelled rule schemas of Section 6.2 and the global resource-preservation statement of Theorem 1. Without it, the preservation theorem would depend on informally assumed rule correctness. With it, the proof obligations are finitely enumerated and each case is independently verifiable. This structure mirrors compiler correctness proofs [13], where local instruction-preservation lemmas are stated case by case before they are composed into a global simulation argument. The explicit case decomposition also delineates the mechanisation target: encoding each case as a goal in Coq or Agda would replace the present paper proof with a mechanically checked one, upgrading the bridge from informal to formal without changing the statement of Theorem 1.

6.3 Resource preservation

Theorem 1 (Resource preservation for certified runs)**.** *Let*

$$(y_0, R_0) \xrightarrow{e_0} (y_1, R_1) \xrightarrow{e_1} \cdots \xrightarrow{e_{n-1}} (y_n, R_n)$$

be a finite execution of a swarm coalgebra such that every step is certified by a locally sound SELL rule. Write $R_k = (\beta_k, \pi_k, \tau_k, \kappa_k)$, and let M_k be the shared-memory component of y_k. Then for every agent $i \in V$,

$$\sum_{k=0}^{n-1} c_i(e_k) \le \beta_0(i) + \sum_{k=0}^{n-1} \rho_i(e_k).$$

Moreover, shared memory grows by provenance-preserving extension, every auditable tool call, handoff, message, shared-memory update, certification, or answer in the run has a corresponding fresh trace item in τ_n, and every claim used by a final answer belongs to the certified-claim component available at the step where the answer is emitted and therefore to κ_n with a supporting shared-memory key.

Proof. Fix an agent $i \in V$. By the budget condition in Definition 8, every certified step satisfies

$$\beta_{k+1}(i) = \beta_k(i) - c_i(e_k) + \rho_i(e_k).$$

We prove by induction on m, for $0 \le m \le n$, that

$$\beta_m(i) = \beta_0(i) - \sum_{k=0}^{m-1} c_i(e_k) + \sum_{k=0}^{m-1} \rho_i(e_k).$$

For $m = 0$, both sums are empty, so the right-hand side is $\beta_0(i)$, which equals the left-hand side $\beta_0(i)$. Assume the equality holds for m. Then

$$\begin{aligned}
\beta_{m+1}(i) &= \beta_m(i) - c_i(e_m) + \rho_i(e_m) \\
&= \beta_0(i) - \sum_{k=0}^{m-1} c_i(e_k) + \sum_{k=0}^{m-1} \rho_i(e_k) - c_i(e_m) + \rho_i(e_m) \\
&= \beta_0(i) - \sum_{k=0}^{m} c_i(e_k) + \sum_{k=0}^{m} \rho_i(e_k).
\end{aligned}$$

Thus the equality holds for all m, in particular for $m = n$. Since the non-negativity condition gives $\beta_n(i) \in \mathbb{N}$, we have $\beta_n(i) \ge 0$. Rearranging the equality for $m = n$ yields

$$\sum_{k=0}^{n-1} c_i(e_k) \le \beta_0(i) + \sum_{k=0}^{n-1} \rho_i(e_k).$$

This proves the budget preservation claim.

For shared memory, local soundness gives $M_k \sqsubseteq M_{k+1}$ at every step. By induction on k, $M_0 \sqsubseteq M_n$. If a key first appears at step k, the shared-memory condition records provenance containing the event and its trace item; extension never changes that binding. Thus every source key available at the end of the run has stable provenance back to the step that introduced it.

For traces, let k be a step at which e_k is auditable. By local soundness, there exists $q_k \in \tau_{k+1} \setminus \tau_k$. The trace condition also gives $\tau_k \subseteq \tau_{k+1}$ for every step, so by induction on the suffix length,

$$\tau_{k+1} \subseteq \tau_n.$$

Hence $q_k \in \tau_n$. If $k < \ell$ are two auditable steps and $q_k = q_\ell$, then $q_k \in \tau_\ell$ by monotonicity, contradicting $q_\ell \in \tau_{\ell+1} \setminus \tau_\ell$. Therefore different auditable events receive distinct trace items.

For final answers, suppose e_k is an answer citing the set of claims K_{e_k}. The answer condition gives, for every $a \in K_{e_k}$, a key k_a such that $(a, k_a) \in \kappa_k$. Certification monotonicity gives $\kappa_k \subseteq \kappa_n$ by induction over the remaining steps. Therefore $(a, k_a) \in \kappa_n$ for every cited claim. This proves the final-answer claim. The certification condition additionally ensures that any claim newly entering κ during the run is backed by a source in the corresponding shared-memory state. By the shared-memory argument, that source has stable provenance. Hence certified answers cannot rely on claims introduced without source support. □

Corollary 1 (Prefix safety)**.** *If an infinite execution has the property that every finite prefix is certified by locally sound SELL rules, then every finite prefix satisfies the budget, graph, permission, provenance, certification, and audit conclusions of Theorem 1.*

Proof. Apply Theorem 1 to an arbitrary finite prefix. Since the prefix was arbitrary, the property holds for all finite prefixes. □

For a fixed raw coalgebra C and initial state $x_0 = (y_0, R_0)$, let $\mathcal{L}_{\mathrm{raw}}(C, x_0)$ be the set of finite words

$$(o_0, e_0) \cdots (o_{n-1}, e_{n-1})$$

generated by raw transitions of C. Let $\mathcal{L}_{\mathsf{cert}}(C, \Sigma_\Gamma, x_0)$ be the subset generated by the certified transition relation $\Longrightarrow_{\mathsf{cert}}$ from the same initial state.

Proposition 4 (Certified safety sublanguage)**.** *For every C, Σ_Γ, and initial state x_0,*

$$\mathcal{L}_{\mathsf{cert}}(C, \Sigma_\Gamma, x_0) \subseteq \mathcal{L}_{\mathrm{raw}}(C, x_0).$$

Moreover, $\mathcal{L}_{\mathsf{cert}}(C, \Sigma_\Gamma, x_0)$ is closed under finite prefixes, certified paths compose whenever the terminal state and ledger of the first path are the initial state and ledger of the second, and every certified word satisfies the budget, provenance, certification, and audit conclusions of Theorem 1.

Proof. The inclusion follows immediately from the definition of $\Longrightarrow_{\mathsf{cert}}$: a certified step exists only when the corresponding raw coalgebra step exists and its event has a derivable certificate. Prefix closure follows because every prefix of a finite sequence of certified steps is again a finite sequence of certified steps beginning at x_0. Composition is concatenation of derivation sequences: if the final state and ledger of the first sequence match the initial state and ledger of the second, the conclusions of the first provide exactly the premises for starting the second. Finally, each certified word is a finite certified run, so Theorem 1 applies to it. Thus certification defines a safety sublanguage of raw behaviour rather than an unrelated execution model. □

Remark 2. *The theorem is conditional on explicit local soundness assumptions. This is the appropriate level of abstraction for the article: a full proof-search implementation of SELL is not required, but each executable transition rule must be shown to satisfy the stated resource, graph, permission, provenance, trace, and certification invariants. Corollary 1 and Proposition 4 also clarify the role of finiteness: the proved theorem is finite-run, but the safety property it establishes is prefix-closed and therefore meaningful for long-running systems monitored one finite prefix at a time.*

Proposition 4 may appear to follow from definitions, but it records three properties that are jointly non-trivial. First, the certified sublanguage is prefix-closed*: any truncation of a certified word remains certified, so a monitor may stop at any step and still hold a valid certificate. Second, certified paths* compose*: if the terminal ledger of one certified segment equals the initial ledger of the next, their concatenation is certified, supporting modular certification of sequential sub-tasks. Third, taken together these properties make the certified sublanguage a* safety language *in the classical sense of Alpern and Schneider [45]: it is characterised by a prefix-closed bad-prefix condition, and violation can always be detected at a finite prefix. This structural fact is what makes the certification discipline compatible with incremental monitoring: a runtime checker can inspect each arriving finite prefix independently without access to future transitions. Corollary 1 makes this operationally precise for infinite executions.*

7 Running Example

The definitions above are parameterised by the agent set V and the interaction graph Γ. The following three-agent system is a finite running instance, not the definition of a swarm. It is used as a concrete witness for the coalgebraic events, SELL-labelled resources, and checker artefacts.

Consider a research assistant swarm with three agents:

$$V = \{\mathsf{Planner}, \mathsf{Retriever}, \mathsf{Verifier}\}.$$

The planner decomposes a research question, the retriever calls bibliographic tools, and the verifier checks whether claims are supported by sources. The interaction graph contains

$$\mathsf{Planner} \to \mathsf{Retriever}, \qquad \mathsf{Retriever} \to \mathsf{Verifier}, \qquad \mathsf{Verifier} \to \mathsf{Planner}.$$

The finite checker instance assigns budgets

$$\beta_0(\mathsf{Planner}) = 3, \qquad \beta_0(\mathsf{Retriever}) = 5, \qquad \beta_0(\mathsf{Verifier}) = 2,$$

permissions for BibliographicSearch and ClaimCheck, empty shared memory M_0, empty trace set τ_0, and empty certified-claim relation κ_0. Thus

$$R_0 = (\beta_0, \pi_0, \tau_0, \kappa_0), \qquad M_0 = \varnothing, \qquad \tau_0 = \varnothing, \qquad \kappa_0 = \varnothing.$$

The accepted trace `valid_research_swarm.json` then gives the certified execution in Table 3.

For example, the second row instantiates the certified transition relation. With $q_2 = \texttt{tr-002}$, $k = \texttt{src\_rutten2000}$, and provenance $p = (\mathsf{Retriever}, \mathsf{BibliographicSearch}, q_2)$, the raw call is admitted as

$$(y_1, R_1) \overset{\mathsf{call}(\mathsf{Retriever},\mathsf{BibliographicSearch},q_2),\, o_2}{\Longrightarrow}_{\mathsf{cert}} (y_2, R_2).$$

The ledger and memory update are exactly

$$\beta_2(\mathsf{Retriever}) = 3, \qquad \tau_2 = \tau_1 \cup \{q_2\}, \qquad M_2 = M_1[k \mapsto (v, p)],$$

while π and κ are unchanged. This concrete transition shows how the abstract judgement $e : (y, R) \rightsquigarrow (y', R')$ is instantiated by the running ledger.

Step	Event	Certificate premise	Ledger effect
1	handoff $P \to R$	graph edge, fresh trace	τ gains `tr-001`; budgets unchanged
2	search call by R	permission, budget $5 \geq 2$, fresh trace	$\beta(R) = 3$; M gains `src_rutten2000` with provenance
3	message $R \to V$	graph edge, fresh trace	τ gains `tr-003`; memory unchanged
4	claim check by V	permission, budget $2 \geq 1$, fresh trace	$\beta(V) = 1$; no new source key
5	certify (a, k)	authorised certifier, existing key k	κ gains (a, k), with $k = \texttt{src\_rutten2000}$
6	handoff $V \to P$	graph edge, fresh trace	certified payload returns to planner
7	answer citing a	claim a already in κ	final answer is backed by k; all trace identifiers are distinct

Table 3: Step-by-step ledger view of the running research-swarm example. Here P, R, and V abbreviate Planner, Retriever, and Verifier.

In the coalgebraic semantics, the seven rows form a path in the global raw swarm coalgebra. In the certified LTS, the same path exists only because every row has a matching SELL-labelled

rule instance. The contrast with the negative traces is direct: if the planner attempts to call BibliographicSearch, the reusable permission premise is absent; if the verifier certifies a missing key, the shared-source premise is absent; if an answer cites an uncertified claim, the answer premise is absent. This is the intended division of labour: coalgebra describes the observable execution path, while SELL specifies which resources justify each step.

8 Prototype Trace Checker

To make the proposed discipline executable, we implemented a lightweight trace checker. The checker is located at `prototype/checker.py` in the accompanying repository [46]; it reads the finite swarm specification, validates the JSON traces, and writes the reproducible summaries used in this section. The generated artefacts are the JSON summary, the CSV summary, the LaTeX table input, a coverage summary, a coverage table, and a checksum manifest.

The prototype is not intended to be a complete SELL proof-search engine. Instead, it implements a decidable instance of the local-soundness conditions in Definition 8. For each event, the implemented fragment checks whether:

- the acting agent is known;
- handoffs and messages respect the interaction graph;
- tool calls are authorised by tool permissions;
- linear budgets are sufficient and are consumed exactly once;
- shared-memory writes are append-only and record provenance;
- final answers use only certified claims;
- tool calls, handoffs, messages, shared-memory updates, certifications, and answers have audit traces.

Definition 9 (Abstract finite trace checker)**.** *For a finite swarm specification $\mathcal{S}$ with agents, graph edges, tools, costs, permissions, certifiers, initial budgets, and auditable event kinds, define*

$$\mathsf{TraceCheck}_{\mathcal{S}} : \mathsf{JSONTrace}_{\mathcal{S}}^{*} \longrightarrow (\{\mathsf{Accept}\} \times \mathsf{CheckState}) + (\{\mathsf{Reject}\} \times \mathsf{Diag}^{*}).$$

The function processes a finite event list $T = e_1 \cdots e_n$ from left to right. Its state is $(\beta, M, \kappa, \tau, D)$, consisting of the current budgets, shared memory, certified claim-source pairs, trace identifiers, and diagnostics. It starts from $(\beta_0, \varnothing, \varnothing, \varnothing, \varnothing)$. At step r, it applies the corresponding one of the six certificate clauses: tool calls require a known agent, known tool, permission, sufficient budget, fresh audit item, exact budget decrement, and append-only writes; shared-memory updates require a known writer, fresh shared keys, admissible scope, provenance, and fresh audit item; handoffs and messages require known endpoints, a graph edge, policy admissibility, and fresh audit item; certifications require an authorised certifier, a pre-existing source key, and fresh audit item before adding (a, k) to κ; answers require every cited claim a to have some $(a, k) \in \kappa$ and a fresh audit item. Any failed premise appends a diagnostic to D. The result is Accept *with the final state iff D is empty, and* Reject *with diagnostics otherwise.*

We write $\mathsf{TraceCheck}_{\mathcal{S}}(T) = \mathsf{Accept}$ as shorthand for saying that the first component of the result is Accept.

The file `checker.py` (see [46]) is the reference implementation of $\mathsf{TraceCheck}_{\mathcal{S}}$ for the JSON schema used in the artefact. The following propositions are stated about the abstract function TraceCheck; the Python program is evidence of reproducibility, not the mathematical object on which the proof depends. The file `rules.json` in the same repository is the concrete finite instance of the abstract specification $\mathcal{S}$; it is the object the checker reads at runtime, not a hypothesis in the mathematical sense.

Proposition 5 (Soundness of accepted prototype traces)**.** *For the finite swarm specification $\mathcal{S}$ encoded by `rules.json` in [46], every trace T such that $\mathsf{TraceCheck}_{\mathcal{S}}(T) = \mathsf{Accept}$ determines a finite certified execution satisfying the local-soundness clauses of Definition 8, with costs, permissions, graph edges, shared-memory keys, provenance records, certified claims, and trace items interpreted exactly as in $\mathcal{S}$.*

Proof. The function $\mathsf{TraceCheck}_{\mathcal{S}}$ processes a trace from left to right while maintaining a ledger consisting of budgets, shared-memory keys with provenance records, certified claims, and trace identifiers. We show that, if no diagnostic is reported, every processed event satisfies the corresponding local-soundness clause.

First, for every auditable event, the trace clause checks that a trace identifier is present and has not previously appeared. Thus the transition from the current trace set τ to the next trace set τ' satisfies $\tau \subseteq \tau'$, and for auditable events there is a fresh $q \in \tau' \setminus \tau$.

Second, if the event is a tool call, the tool-call clause verifies that the acting agent is known, that the tool exists, that the agent belongs to the tool's allowed-agent list, and that the current budget is at least the tool cost. If these checks pass, $\mathsf{TraceCheck}_{\mathcal{S}}$ subtracts exactly that cost from the agent's budget and leaves other agents' budgets unchanged. Hence the permission and budget clauses of Definition 8 hold with $\rho_i(e) = 0$ for the current prototype. Shared-memory writes are processed by the memory-write clause, which rejects missing keys, unsupported scopes, and attempts to overwrite an existing shared key; accepted writes therefore extend memory monotonically and attach the event, step, trace item, and value as provenance.

Third, if the event is a handoff or message, the corresponding graph clause verifies that both endpoint agents are known and that the directed edge belongs to the interaction graph. Hence the graph condition holds.

Fourth, if the event certifies a claim, the certification clause verifies that the certifying agent is authorised and that the cited source is already present in shared memory before adding the claim to the certified-claim set. Hence every newly certified claim is source-supported. If the event is a final answer, the answer clause verifies that each cited claim is already certified, so the answer condition holds.

The abstract checker records a diagnostic whenever one of these obligations fails. Therefore an accepted trace has no failed local-soundness obligation. Reading each accepted event as the corresponding certified transition yields the required finite certified execution. □

The following two tables are produced by running `checker.py` from the repository [46] against the

full conformance suite. They are included verbatim as generated LaTeX fragments; the generating command and SHA-256 checksums are given in Section 9.

Trace	Status	Events	Cost	Diagnostic
invalid_bad_memory_scope	rejected	1	0	step 1: unsupported memory scope 'global'
invalid_budget_violation	rejected	3	4	step 3: insufficient budget for 'Retriever': required 2, available 1
invalid_certificate_source	rejected	1	0	step 1: certificate source 'src_missing' not found in shared memory
invalid_duplicate_trace	rejected	2	0	step 2: duplicate trace id 'tr-dup'
invalid_handoff_violation	rejected	1	0	step 1: handoff edge 'Planner -> Verifier' is not allowed
invalid_message_violation	rejected	1	0	step 1: message edge 'Planner -> Verifier' is not allowed
invalid_missing_trace	rejected	1	2	step 1: event 'tool_call' lacks required audit trace
invalid_shared_memory_overwrite	rejected	2	4	step 2: shared memory key 'src_same' already exists
invalid_tool_permission	rejected	1	2	step 1: agent 'Planner' lacks permission for tool 'BibliographicSearch'
invalid_unauthorized_certifier	rejected	2	2	step 2: agent 'Planner' is not authorised to certify claims
invalid_uncertified_answer	rejected	1	0	step 1: answer uses uncertified claim 'unsupported_claim'
invalid_unknown_agent	rejected	1	0	step 1: unknown source agent 'Ghost'
invalid_unknown_tool	rejected	1	0	step 1: unknown tool 'NonexistentTool'
valid_claim_reuse_trace	accepted	7	2	OK
valid_langgraph_like_trace	accepted	7	3	OK
valid_memory_update_reuse_trace	accepted	6	0	OK
valid_research_swarm	accepted	7	3	OK
valid_two_source_research_swarm	accepted	9	5	OK

Table 4: Results produced by the prototype trace checker. Accepted traces satisfy the graph, permission, budget, provenance, certification, and audit-trace constraints. Rejected traces illustrate violations of linear budget, certificate support, trace freshness, handoff topology, audit obligations, shared-memory append-only discipline, tool permission, and answer certification.

Metric	Value
Total traces	18
Accepted traces	5
Rejected controls	13
Accepted event types	answer, certify, handoff, message, shared_memory_update, tool_call
Accepted framework styles	langgraph-like
Accepted events	36
Accepted budget consumed	13

Table 5: Coverage summary for the finite conformance suite. The table reports suite size, accepted behaviours, exercised event kinds, and aggregate accepted-resource usage.

The accepted traces are minimal research-assistant conformance scenarios of the kind described above. They now include a one-source trace, a two-source trace, a shared-memory-update trace, a claim-reuse trace showing that certified claims are reusable while audit identifiers remain fresh, and a LangGraph-like normalised trace carrying framework-style metadata. The rejected traces are negative controls covering budget overuse, unsupported certification, duplicate trace identifiers, illegal handoffs, missing audit evidence, shared-memory overwrite, unauthorised tool use, and answers that cite uncertified claims. Thus the artefact operationalises the formal claim behind Theorem 1: by Proposition 5, accepted traces instantiate local soundness and therefore inherit graph, permis-

sion, budget, provenance, certification, and audit invariants. The suite is intentionally a finite conformance check, not evidence of empirical scalability.

Table 6 makes the correspondence explicit. Each formal obligation is discharged by one of the certificate schemas, implemented by a checker routine, and exercised by at least one positive or negative trace.

Formal obligation	Certificate rule	Checker routine	Exercising traces
Graph edge for handoffs/messages	handoff, message	`check_handoff`, `check_message`	`valid_research_swarm`; `invalid_handoff_violation`, `invalid_message_violation`
Tool permission and known tool	call	`check_tool_call`	`valid_two_source_research_swarm`; `invalid_tool_permission`, `invalid_unknown_tool`
Linear budget consumption	call	`check_tool_call`	`valid_research_swarm`; `invalid_budget_violation`
Append-only source memory	writeShared	`record_writes`	`valid_research_swarm`; `invalid_shared_memory_overwrite`, `invalid_bad_memory_scope`
Fresh audit evidence	all auditable rules	`require_trace`	`valid_research_swarm`; `invalid_missing_trace`, `invalid_duplicate_trace`
Claim-source certification	certify	`check_certify`	`valid_two_source_research_swarm`; `invalid_certificate_source`, `invalid_unauthorized_certifier`
Answer cites certified claims	answer	`check_answer`	`valid_research_swarm`; `invalid_uncertified_answer`
Known participating agents	all agent-indexed rules	`require_known_agent`	valid traces; `invalid_unknown_agent`

Table 6: Correspondence between formal obligations, certificate rules, checker routines, and conformance traces.

For the implemented fragment, TraceCheck is not only a post-hoc validator; it is an executable recogniser for derivations in the finite certificate calculus.

Proposition 6 (Adequacy of the implemented checker fragment)**.** *Fix the finite specification $\mathcal{S}$ encoded by `rules.json` in [46] and restrict events to the six certificate schemas above, with no budget replenishment. For any finite JSON trace T, if $\mathsf{TraceCheck}_{\mathcal{S}}(T) = \mathsf{Accept}$, then there exists a sequence π_T of SELL-labelled certificate-rule instances deriving the certified execution represented by T. Conversely, if a trace over the implemented JSON schema is generated by such a sequence of rule instances, then $\mathsf{TraceCheck}_{\mathcal{S}}(T) = \mathsf{Accept}$.*

Proof. For the forward direction, read the accepted trace from left to right. Proposition 5 shows that every accepted event satisfies the premises of its corresponding certificate schema: known agents and graph facts for handoffs and messages, permission and sufficient budget for tool calls, fresh shared keys for memory writes, source support and certifier authority for certifications, certified-claim availability for answers, and fresh trace identifiers for auditable events. Instantiate that schema at each step with the current ledger maintained by $\mathsf{TraceCheck}_{\mathcal{S}}$. The resulting sequence of rule instances is π_T, and its conclusions produce exactly the next ledger state computed by $\mathsf{TraceCheck}_{\mathcal{S}}$.

For the converse direction, assume that a trace is generated by a sequence of implemented certificate-rule instances. Each rule premise is one of the predicates checked by the corresponding clause of Definition 9. Since the rule instance is valid, $\mathsf{TraceCheck}_{\mathcal{S}}$ finds the required graph edge, permission, budget, source key, certified claim, or fresh trace identifier condition satisfied. The rule conclusion

and TraceCheck update agree on budgets, shared-memory extension, certified-claim insertion, and trace insertion. Induction over the sequence length therefore shows that no diagnostic is produced and that the maintained abstract state coincides with the ledger obtained from the derivation. Hence $\mathsf{TraceCheck}_{\mathcal{S}}$ accepts the trace. □

The checker can be connected to existing agent frameworks by a trace-normalisation adapter rather than by changing the semantics. An AutoGen conversation log, a LangGraph execution state, or an OpenAI Agents SDK trace can be mapped into the prototype schema by extracting agent names, event kinds, directed handoffs or messages, tool invocations, memory writes, trace identifiers, and final-answer citations. This paper does not claim an empirical evaluation on those frameworks; the point is an interoperability path: framework traces become candidate raw executions, and the checker certifies the resource discipline when the required fields are present.

A stylised framework-normalisation step is shown in Table 7. It is not an additional experiment; it records the shape of the adapter assumed by the semantics. A LangGraph-style node transition or AutoGen message that invokes a tool is treated as a raw event first. Only after the permission, budget, memory, and audit fields are present can it become a certified event.

Framework-level field	Normalised event field	Certificate obligation
node or speaker name `Retriever`	agent = Retriever	known agent and local context
tool invocation `BibliographicSearch`	event = tool_call, tool = BibliographicSearch	reusable tool permission and linear budget
run/span identifier `tr-002`	trace = q_2	fresh audit evidence
state write `src_rutten2000`	shared write (k, v, p)	append-only source memory with provenance
edge `Retriever -> Verifier`	message or handoff target Verifier	graph admissibility

Table 7: Stylised normalisation of a framework trace step into the certificate schema.

9 Assumptions, Reproducibility, and Limitations

The preceding sections separate three layers: raw coalgebraic behaviour, SELL-labelled certification, and executable trace checking. This section records the assumptions under which the results hold, explains the role of the preservation theorem, and states the limits of the artefact. The purpose is to make the claims auditable rather than to enlarge them.

9.1 Formal assumptions

The deterministic core of the paper relies on the following assumptions.

Assumption 1: the base category is Set. States, observations, raw events, memories, ledgers, and traces are treated as sets, and the functors used in Propositions 1 and 2 are endofunctors on **Set**. This conservative choice keeps the proof obligations explicit. Other bases, such as **Rel** or a Kleisli category for probability, are meaningful extensions but are not used in the deterministic preservation theorem.

Assumption 2: the event functor is deterministic. The swarm functor is

$$S_\Gamma(X) = (\mathsf{RawEv}_\Gamma \times X)^{\mathsf{Obs}_\Gamma}.$$

Thus a global state and a global observation determine one raw event and one next global state. The local agent interface

$$c_i : X_i \to (\mathsf{Ev}_i \times X_i)^{\mathsf{Obs}_i}$$

is the local shape assembled into the global swarm coalgebra. A probabilistic variant would replace $\mathsf{RawEv}_\Gamma \times X$ by $\mathcal{D}(\mathsf{RawEv}_\Gamma \times X)$, but no theorem in this paper depends on that variant.

Assumption 3: SELL is used as a certification discipline. The paper fixes a concrete subexponential preorder $\preceq_\Gamma$, a labelled context shape $\Delta; \Theta$, and rule schemas for tool calls, memory writes, handoffs, messages, certification, and answers. These ingredients use SELL-labelled contexts to control which resources are reusable, local, consumable, or auditable. Table 2 records the operational rule-to-obligation correspondence, and Lemma 1 proves local soundness for the six schemas. The checker is not a full SELL proof-search engine. Instead, Definition 8 specifies the soundness obligations that any implementation of the rules must discharge: graph preservation, permission checking, budget accounting, shared-memory provenance, trace freshness, certification monotonicity, and answer safety. Proposition 6 proves adequacy only for the finite implemented fragment. A full sequent-calculus implementation or proof-assistant mechanisation could replace the checker, provided these obligations are proved for each rule.

The relation between this fragment and full SELL is one of *soundness, not completeness*: every accepted trace has a SELL-labelled derivation in the signature Σ_Γ, but the checker does not enumerate all valid SELL derivations. Completeness with respect to full SELL would require a decision procedure for the relevant sequent-calculus fragment; such procedures exist for restricted subexponential systems [27], but their complexity exceeds what a linear-pass checker can achieve. The deliberate choice of soundness-without-completeness keeps the implemented fragment precisely delineated: Proposition 6 establishes exact correspondence for the six implemented rules, while any extension can be integrated by adding one new rule schema and verifying its local-soundness clause separately, without disrupting existing certificates.

Assumption 4: the preservation theorem is finite-run and safety-oriented. The behavioural results are stated over non-empty finite observation words, and Theorem 1 is stated for finite certified executions. The theorem does not prove liveness, optimality, convergence, or fairness. It does prove a prefix-closed safety property: by Corollary 1, any long-running execution whose finite prefixes remain certified preserves the stated graph, permission, budget, provenance, certification, and audit invariants on every finite prefix.

Assumption 5: generic semantics and finitary checking are distinct. The final-coalgebra arguments are generic for the Set-functors used in the deterministic core. The prototype is finitary: it uses a finite set of agents, a finite tool catalogue, finite budgets, and finite JSON traces. At the semantic level, Propositions 2 and 3 apply to arbitrary sets of observations and raw events. At the executable level, Proposition 5 applies to the finite specification in `rules.json` [46]. The prototype illustrates and checks the theory; it does not replace the theory.

9.2 Role of the preservation theorem

The preservation theorem is intentionally not a deep theorem about all of SELL. Its role is to connect local certificates with global executions. Each certified transition must satisfy local obligations: the graph edge must exist, the tool must be permitted, the budget update must be exact, shared-memory writes must carry provenance, audit identifiers must be fresh, and final answers may cite only previously certified claims. Theorem 1 then proves that these local obligations compose over arbitrary finite runs. Consequently, a violation cannot be introduced merely by sequencing individually certified steps.

This local-to-global form is useful because raw coalgebraic behaviour and certified behaviour are intentionally different. The coalgebra describes what an implementation may emit; the certification layer identifies which emitted transitions are acceptable under the resource policy. The theorem states that the certified sub-behaviour has stable global invariants, even though the raw coalgebra may contain unauthorised, unaffordable, or unaudited transitions.

Proposition 7 (Deterministic core under the stated assumptions)**.** *Under Assumptions 1–5, every raw deterministic swarm coalgebra has a unique observable behaviour map, bisimilar states have identical observable behaviours, certified paths form a prefix-closed safety sublanguage of raw behaviours, every locally sound certified finite run satisfies resource and provenance preservation, every certified long-running execution is prefix-safe, every accepted prototype trace instantiates the local-soundness conditions for the finite conformance scenario, and accepted traces in the implemented fragment correspond to SELL-labelled certificate derivations.*

Proof. The first claim follows from Proposition 2. The second claim follows from Proposition 3. The safety-sublanguage claim follows from Proposition 4. Resource preservation is Theorem 1. Prefix safety follows from Corollary 1. The prototype local-soundness claim follows from Proposition 5, and the derivation correspondence follows from Proposition 6. These results cover the semantic, behavioural, resource, and executable layers of the deterministic framework. □

9.3 Reproducibility artefact

The artefact in `prototype/` is a conformance suite for the stated proof obligations, not a statistical benchmark of agent performance. It consists of `prototype/rules.json`, the trace files in `prototype/traces/`, the checker `prototype/checker.py`, and the generated outputs in `prototype/results/`; all files are publicly available at [46]. The checker uses the Python standard library only; Python 3.10 or newer is recommended. After cloning the repository, the complete regeneration command is:

```
git clone https://github.com/cero1979/coalgebraic-agent-swarms
cd coalgebraic-agent-swarms
python3 prototype/checker.py \
  --rules prototype/rules.json \
  --traces prototype/traces/*.json \
  --out-dir prototype/results
```

The expected conformance summary is:

- 18 traces total;
- 5 accepted traces: `valid_research_swarm.json`, `valid_two_source_research_swarm.json`, `valid_memory_update_reuse_trace.json`, `valid_claim_reuse_trace.json`, and `valid_langgraph_like_trace.json`;
- 13 rejected negative controls.

The command writes `results.json`, `results.csv`, `results_table.tex`, `coverage_summary.json`, `coverage_table.tex`, and `MANIFEST.json` under `prototype/results/`. For the version archived in the repository [46], the SHA-256 checksums are:

File	SHA-256
`results.json`	`1d3da093af3bdae68a8ff549978f9e00 b96a8e547fa4f7629d75f3e5d57d9d42`
`coverage_summary.json`	`3a989449eb320c771f498be7fc21416c cebdd0a9669c53ee62c34ad66821f3d1`
`MANIFEST.json`	`768593b44bb8134e085090af077830a2 30d6a17c60d89427310e487f35e91936`

The generated LaTeX files `prototype/results/results_table.tex` and `prototype/results/coverage_table.tex` are included as Tables 4 and 5.

The accepted traces exercise minimal certified research-assistant conformance scenarios, including separate shared-memory update, reusable certified-claim evidence, and LangGraph-like trace-normalisation metadata. The rejected traces exercise budget overuse, illegal handoff, missing audit evidence, duplicate trace identifiers, unsupported certificates, shared-memory overwrite, unauthorised tool use, and uncertified answers.

The artefact supports the formal development in four ways. First, it gives a concrete finite instance of the abstract ledger $R = (\beta, \pi, \tau, \kappa)$. Second, it makes local-soundness failures observable as trace rejections. Third, accepted traces instantiate Proposition 5, and therefore inherit Theorem 1. Fourth, Proposition 6 identifies the checker as an executable recogniser for the implemented certificate fragment. This is evidence that the interface is executable, but it is not evidence of scalability, usability, or superiority over existing agent frameworks.

9.4 Threats to validity and limitations

The main limitations are as follows.

1. The SELL layer is a labelled certification calculus rather than a complete proof-search implementation. The paper proves adequacy only for the implemented fragment; a mechanised development would need to encode the subexponential signature, rule schemas, and local-soundness lemmas directly.
2. Shared source memory is append-only in the deterministic core. This is appropriate for audit evidence, but it does not model destructive update, deletion, conflict resolution, or mutable heap ownership. Those features could be added using versioned keys, explicit revocation events, temporal provenance records, or a separation-logic-style ownership layer for mutable private state.

3. The core theory is deterministic. Probabilistic policies, partial observability, reinforcement learning, MDP/POMDP objectives, and stochastic bisimulation require the distributional extension mentioned earlier but not developed here.
4. The prototype uses a hand-written conformance suite supplemented by targeted negative mutations. It is sufficient to test the proof obligations implemented by the checker, but it is not a broad empirical evaluation of multi-agent systems.
5. Interoperability with AutoGen, LangGraph, or the OpenAI Agents SDK is currently a trace-normalisation path rather than an implemented integration. Framework logs can be mapped to the schema when they expose agent names, event kinds, handoffs or messages, tool calls, memory writes, trace identifiers, and final-answer citations; evaluating such adapters is future work.

These limitations are not incidental. They mark the boundary between the present contribution—a compositional semantics and resource-certification discipline for deterministic traces—and the next stages: mechanised SELL proofs, richer memory models, probabilistic coalgebras, and adapters for production framework traces. The restrictions are also what make the deterministic preservation theorem, the certified-sublanguage result, and the checker-adequacy proposition possible: the paper isolates a minimal semantic core on which probabilistic choice, mutable memory, or framework-specific adapters can be added conservatively rather than assumed informally.

10 Discussion

The proposed framework clarifies several distinctions.

First, an agent is not reduced to a cellular automaton. Cellular automata are specific local-update systems; agentic AI systems involve memory, tools, delegation, stochasticity, and external resources. Coalgebra is more general and better suited to the level of abstraction needed here.

Second, SELL is not proposed as a universal logic for all aspects of AI. It is proposed as a resource discipline. If the focus is mutable heap ownership, separation logic may be more appropriate. If the focus is epistemic change, dynamic epistemic logic is central. If the focus is static approximation of possible tool effects, a type-and-effect system may be the right abstraction. If the focus is optimal action under uncertainty, MDPs or POMDPs remain fundamental. SELL is most useful when the central issue is controlled use of assumptions, budgets, permissions, provenance, and trace obligations.

Third, the formal contribution lies in the interface between coalgebra and resource logic. Coalgebra gives a semantics of possible behaviours; SELL turns some behaviours into certified behaviours. In particular, framework guardrails can be understood as operational filters, while the certification calculus records why a transition is admissible and which linear or reusable resources justify it. Likewise, tracing changes role: it is not only a debugging record, but evidence tied to source keys, certified claims, and final answers.

Fourth, the framework explains why handoffs are more than routing edges. The graph condition says that control may move from one agent to another, but the certificate additionally records payload admissibility, trace evidence, and the ledger state available after transfer. Richer responsibility

transfer could be modelled by adding explicit obligations to the handoff rule, for example duties to preserve a source key, revoke a stale claim, or emit a downstream verification trace.

Fifth, the conformance suite is the appropriate form of evaluation for a semantic contribution of this kind. The paper's claim is not that the prototype outperforms existing agent frameworks on throughput, recall, or accuracy benchmarks. The claim is that the certification calculus is executable and that its decidable fragment corresponds exactly to the stated proof obligations. The evidence for this claim is a suite that exercises every obligation with both positive witnesses (accepted traces) and negative controls (rejected traces that each isolate one specific violation), together with a checker-adequacy theorem that closes the gap between the executable fragment and the formal calculus. This methodology parallels that of certifying algorithms [10]: the witness is checked independently of the algorithm that produced it, and it is the correctness of the checker—not the performance of any particular agent system—that gives the suite its evidential value. Evaluating whether real AutoGen, LangGraph, or OpenAI SDK traces can be normalised into the schema would address a different and complementary question; that is explicitly identified as future work in the limitations section.

11 Conclusion

This paper proposed a coalgebraic and resource-sensitive semantics for tool-augmented agent swarms. Individual agents are modelled as coalgebras from observations to events and continuations. Swarms are modelled as global coalgebras indexed by an interaction graph and equipped with shared memory, environment state, and a resource ledger. SELL supplies a labelled proof-theoretic layer for distinguishing persistent memory, shared context, local agent resources, tool permissions, consumable budgets, and audit traces.

The resulting framework offers a path toward formal reasoning about agentic AI systems: behavioural equivalence can compare architectures, handoffs can be treated as typed transitions, and certified runs can satisfy resource-preservation, provenance, and traceability properties. The assumptions and limitations stated above make explicit what is proved in the deterministic core: raw swarm coalgebras have canonical observable behaviours, bisimilar states agree observationally, certified behaviours form a prefix-closed safety sublanguage of raw behaviours, locally sound certified runs preserve the stated resource invariants, accepted prototype traces instantiate those obligations in a finite conformance suite, and the implemented checker fragment is adequate for the corresponding SELL-labelled certificate rules. Natural extensions are proof-assistant mechanisation of local soundness, richer memory models with revocation and versioning, probabilistic coalgebras, systematic mutation-based trace generation, and adapters for traces produced by existing multi-agent frameworks.

The aim is not to replace existing agentic frameworks or empirical evaluations of them. It is to provide a compositional, resource-sensitive semantic layer on which framework traces can be interpreted, certified, and checked against explicit obligations for permissions, budgets, provenance, audit evidence, and answer support.

References

[1] Jean-Yves Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987. DOI: 10.1016/0304-3975(87)90045-4.

[2] Jan J. M. M. Rutten. Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80, 2000. DOI: 10.1016/S0304-3975(00)00056-6.

[3] Bart Jacobs. *Introduction to Coalgebra: Towards Mathematics of States and Observation.* Cambridge University Press, 2016. DOI: 10.1017/CBO9781316823187.

[4] Ana Sokolova. Probabilistic systems coalgebraically: A survey. *Theoretical Computer Science*, 412(38):5095–5110, 2011. DOI: 10.1016/j.tcs.2011.05.008.

[5] Dirk Pattinson. Coalgebraic modal logic: Soundness, completeness and decidability of local consequence. *Theoretical Computer Science*, 309(1–3):177–193, 2003. DOI: 10.1016/S0304-3975(03)00201-9.

[6] Corina Cîrstea, Alexander Kurz, Dirk Pattinson, Lutz Schröder, and Yde Venema. Modal logics are coalgebraic. *The Computer Journal*, 54(1):31–41, 2011. DOI: 10.1093/comjnl/bxp004.

[7] Matthew Hennessy and Robin Milner. Algebraic laws for nondeterminism and concurrency. *Journal of the ACM*, 32(1):137–161, 1985. DOI: 10.1145/2455.2460.

[8] Davide Sangiorgi. *Introduction to Bisimulation and Coinduction.* Cambridge University Press, 2011. DOI: 10.1017/CBO9780511777110.

[9] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking.* MIT Press, 2008.

[10] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. DOI: 10.1016/j.cosrev.2010.09.009.

[11] George C. Necula. Proof-carrying code. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 106–119. ACM, 1997. DOI: 10.1145/263699.263712.

[12] Luís Cruz-Filipe, João Marques-Silva, and Peter Schneider-Kamp. Efficient certified resolution proof checking. In *Tools and Algorithms for the Construction and Analysis of Systems*, LNCS 10205, pages 118–135. Springer, 2017. DOI: 10.1007/978-3-662-54577-5_7.

[13] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. DOI: 10.1145/1538788.1538814.

[14] Alexandru Baltag. A coalgebraic semantics for epistemic programs. *Electronic Notes in Theoretical Computer Science*, 82(1):17–38, 2003. DOI: 10.1016/S1571-0661(04)80630-3.

[15] Corina Cîrstea and Mehrnoosh Sadrzadeh. Coalgebraic epistemic update without change of model. In *Algebra and Coalgebra in Computer Science, CALCO 2007*, Lecture Notes in Computer Science. Springer, 2007.

[16] Facundo Carreiro, Daniel Gorín, and Lutz Schröder. Coalgebraic announcement logics. In *Automata, Languages, and Programming: 40th International Colloquium, ICALP 2013*, pages 101–112. Springer, 2013.

[17] Maja Hadzic and Elizabeth Chang. Using coalgebra and coinduction to define ontology-based multi-agent systems. *International Journal of Metadata, Semantics and Ontologies*, 3(3):197–209, 2008. DOI: 10.1504/IJMSO.2008.023568.

[18] Hans van Ditmarsch, Wiebe van der Hoek, and Barteld Kooi. *Dynamic Epistemic Logic*. Springer, Synthese Library 337, 2007. DOI: 10.1007/978-1-4020-5839-4.

[19] Frank M. V. Feys, Helle Hvid Hansen, and Lawrence S. Moss. Long-term values in Markov decision processes, (co)algebraically. In *Coalgebraic Methods in Computer Science, CMCS 2018*, LNCS 11202, pages 78–99. Springer, 2018. DOI: 10.1007/978-3-030-00389-0_6.

[20] Philip R. Cohen and Hector J. Levesque. Intention is choice with commitment. *Artificial Intelligence*, 42(2–3):213–261, 1990. DOI: 10.1016/0004-3702(90)90055-5.

[21] Yoav Shoham. Agent-oriented programming. *Artificial Intelligence*, 60(1):51–92, 1993. DOI: 10.1016/0004-3702(93)90034-9.

[22] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 1995. DOI: 10.1017/S0269888900008122.

[23] Anand S. Rao and Michael P. Georgeff. BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS 1995)*, pages 312–319, 1995.

[24] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Second edition, John Wiley & Sons, 2009.

[25] Tim Finin, Richard Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management*, pages 456–463. ACM, 1994. DOI: 10.1145/191246.191322.

[26] Foundation for Intelligent Physical Agents. FIPA ACL message structure specification. Standard SC00061G, 2002.

[27] Vivek Nigam and Dale Miller. Algorithmic specifications in linear logic with subexponentials. In *PPDP 2009: Proceedings of the 11th International ACM SIGPLAN Symposium on Principles and Practice of Declarative Programming*, pages 129–140. ACM, 2009. DOI: 10.1145/1599410.1599427.

[28] Joëlle Despeyroux, Carlos Olarte, and Elaine Pimentel. Hybrid and subexponential linear logics. *Electronic Notes in Theoretical Computer Science*, 332:95–111, 2017. DOI: 10.1016/j.entcs.2017.04.007.

[29] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS 2002)*, pages 55–74. IEEE Computer Society, 2002. DOI: 10.1109/LICS.2002.1029817.

[30] John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *POPL 1988: Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 47–57. ACM, 1988. DOI: 10.1145/73560.73564.

[31] Flemming Nielson, Hanne R. Nielson, and Chris Hankin. *Principles of Program Analysis*. Springer, 1999. DOI: 10.1007/978-3-662-03811-6.

[32] Gordon Plotkin and John Power. Algebraic operations and generic effects. *Applied Categorical Structures*, 11(1):69–94, 2003. DOI: 10.1023/A:1023064908962.

[33] Luc Moreau and Paolo Missier, editors. PROV-DM: The PROV data model. W3C Recommendation, 2013.

[34] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory—ICDT 2001*, LNCS 1973, pages 316–330. Springer, 2001. DOI: 10.1007/3-540-44503-X_20.

[35] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009. DOI: 10.1561/1900000006.

[36] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[37] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. DOI: 10.1016/S0004-3702(98)00023-X.

[38] Qingyun Wu et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023. DOI: 10.48550/arXiv.2308.08155.

[39] Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[40] Timo Schick et al. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 36, 2023.

[41] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges. *Vicinagearth*, 1:9, 2024. DOI: 10.1007/s44336-024-00009-2.

[42] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024.

[43] OpenAI. OpenAI Agents SDK documentation. `https://openai.github.io/openai-agents-python/`, accessed May 12, 2026.

[44] LangChain. LangGraph multi-agent and handoff documentation. `https://docs.langchain.com/oss/python/langchain/multi-agent/handoffs`, accessed May 12, 2026.

[45] Bowen Alpern and Fred B. Schneider. Recognizing safety and liveness. *Distributed Computing*, 2(3):117–126, 1987. DOI: 10.1007/BF01782772.

[46] Author omitted for review. Coalgebraic agent swarms: Prototype trace checker and conformance suite [software artefact]. Repository: `https://github.com/cero1979/coalgebraic-agent-swarms`, 2026.